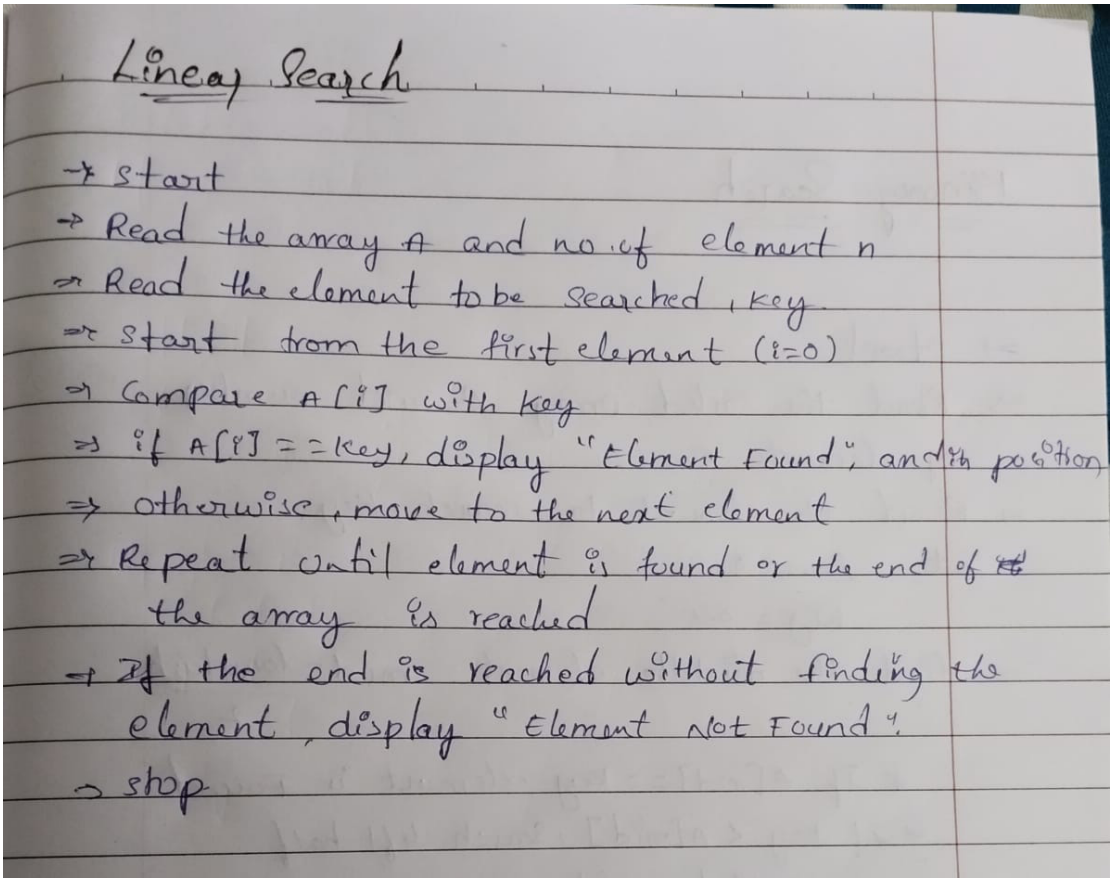
PRACTICAL: 02

Aim: To implement and find the time analysis of Linear Search and Binary Search algorithm.

A) Linear Search:

Algorithm:



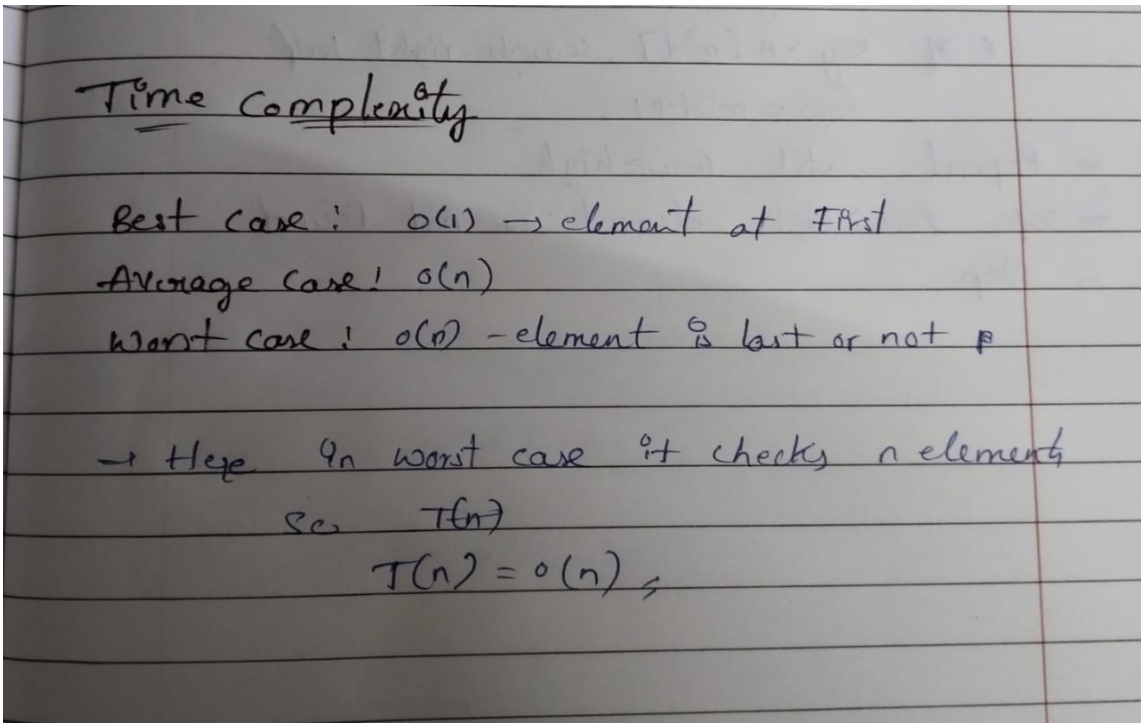
Program / Code:

```
#include <iostream>
using namespace std;
int main() {
   int arr[100],i,n,key;
   cout<<"enter number of array elements:"<<endl;
   cin>>n;
   cout<<"enter array elements:"<<endl;
   for(i=0;i<n;i++){
      cin>>arr[i];
   }
   cout<<"enter element to serach:"<<endl;
   cin>>key;
   for(i=0;i<n;i++){
      if(arr[i]==key)
      {
         cout<<"element found at index of:"<<i;
      }
   }
   return 0;
}
```

Output:

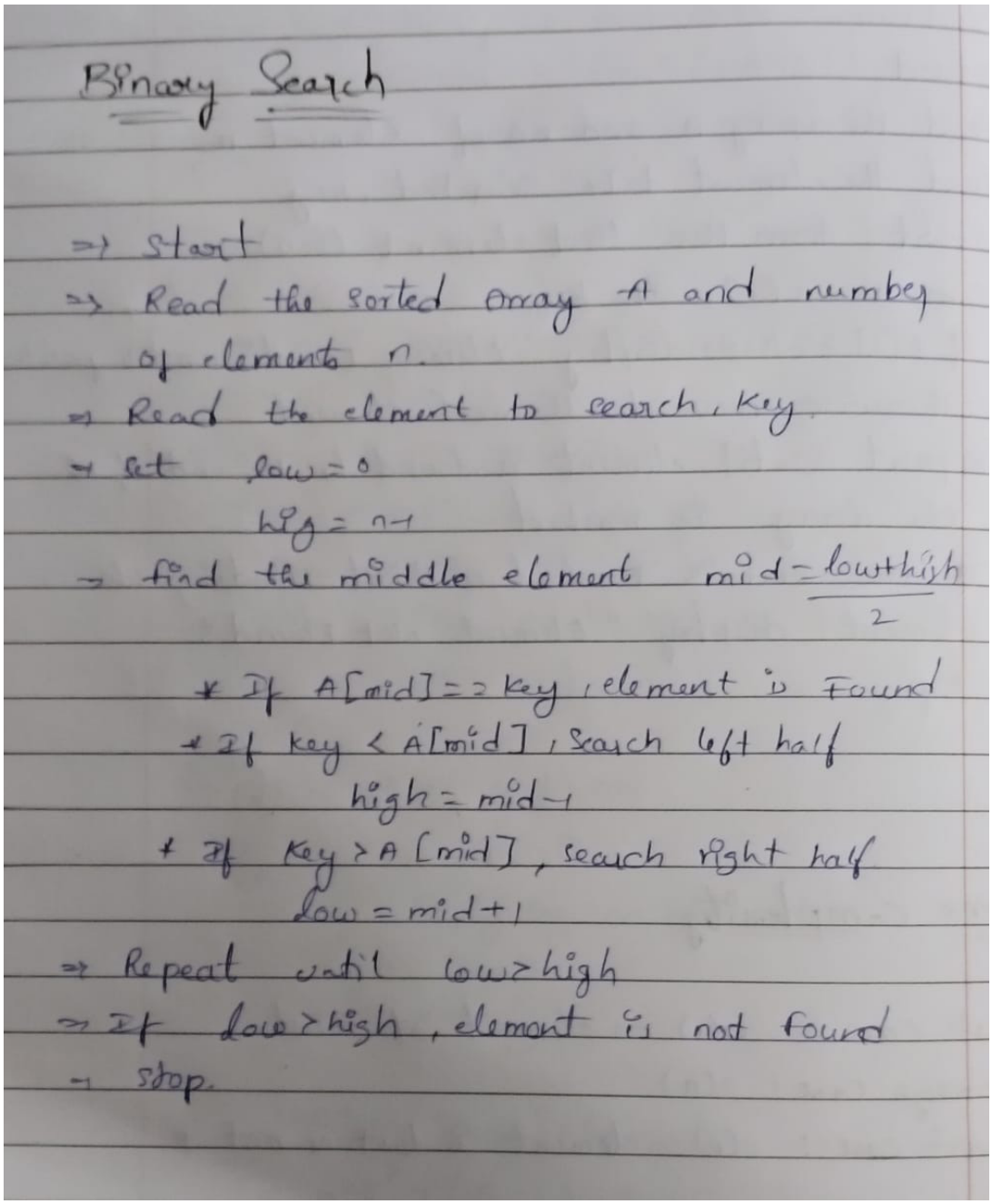
```
  "/Users/lokeshgandreti/Desktop/c++/practical 2/linear1"
● lokeshgandreti@LOKESHs-MacBook-Air practical 2 % "/Users/lokeshgandreti/Desktop/c++/practical 2/linear1"
  Enter the number of elements: 5
  Enter the array elements: 8 3 5 1 9
  Enter element to search:5
  Element Not Found%
○ lokeshgandreti@LOKESHs-MacBook-Air practical 2 %
```

Time Complexity:



B) Binary Search:

Algorithm:



Program / Code:

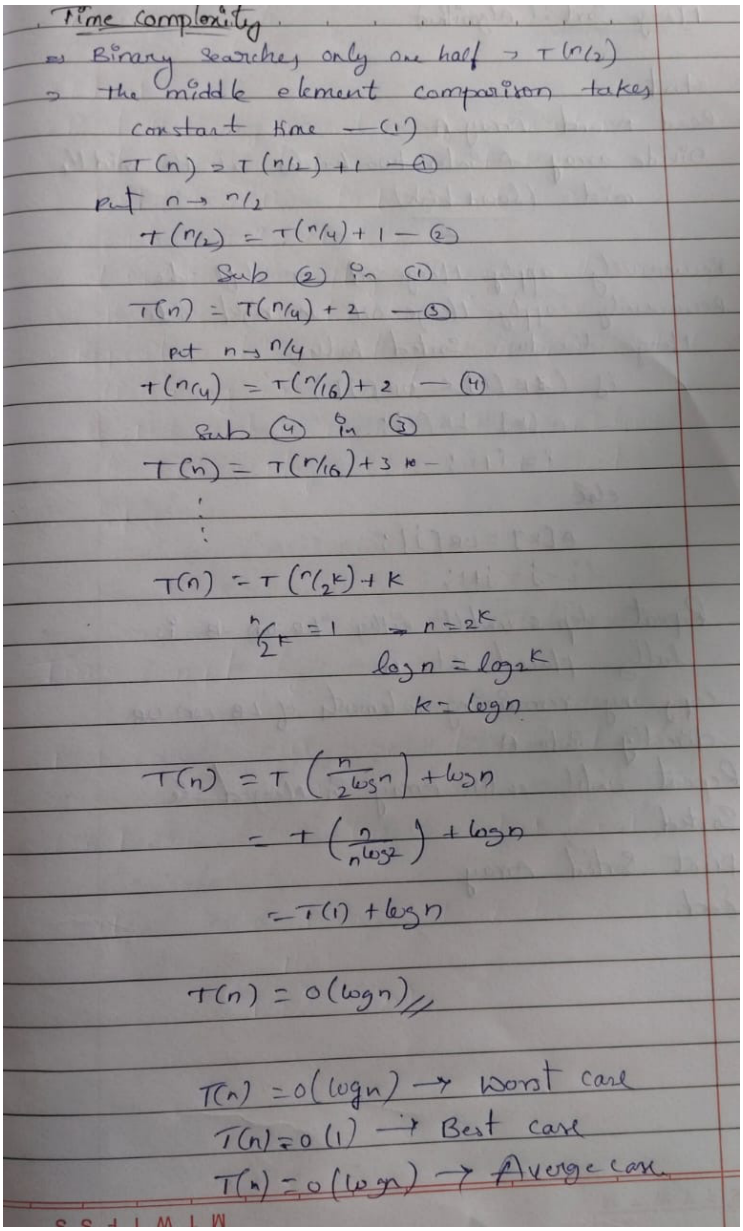
```
#include <iostream>
using namespace std;
void bubblesort(int arr[], int n) {
   for(int i=0; i<n-1; i++) {
      for(int j=0; j<n-i-1; j++) {
         if(arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
         }
      }
   }
}
int binarySearch(int arr[], int n, int key) {
int low = 0, high = n-1; while(low <=
high) {
      int mid = (low + high) / 2; if(arr
      [mid] == key) return mid; else if(arr
      [mid] < key) low = mid+1; else high
      = mid-1;
   }
   return -1;
}
int main() { int arr[100], n, key; cout <<
"Enter number of array elements: ";
```

```
    cin >> n;
    cout << "Enter array elements: ";
    for(int i=0; i<n; i++) {
        cin >> arr[i];
    } 6
    bubblesort(arr, n);
    cout << "Sorted elements: ";
    for(int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "Enter element to search: ";
    cin >> key;
    int result = binarySearch(arr, n, key);
    if(result != -1)
        cout << "Element found at index " << result << endl;
    else
        cout << "Element not found" << endl;
    return 0;
}
```

Output:

```
  "/Users/lokeshgandreti/Desktop/c++/practical 2/binary"
● lokeshgandreti@LOKESHs-MacBook-Air practical 2 % "/Users/lokeshgandreti/Desktop/c++/practical 2/binary"
  Enter the size of array: 6
  Enter 6 elements in sorted order: 2 5 12 9 23 80
  Enter element to search: 12
  Element found at index 2
○ lokeshgandreti@LOKESHs-MacBook-Air practical 2 %
```

Time Complexity:



Conclusion:

In this practical, we studied and implemented two searching algorithms: Linear Search checks each element one by one, so it takes more time as the list size grows. Binary Search is much faster because it works only on sorted lists and cuts the list in half every time, making it a better choice for large data.